st data automatically or manually from backups as stated above.

- We do not expect auth failures while using a centralized model with users/accounts setup within NATS at deployment-time but the introduction of an external auth callout service can add further failure domains to the system. As a dependency, we'll need to guarantee reliability SLOs on our implementation of an auth-server to be equal or higher than those for the NATS service itself. The development of such an abstraction is out-of-scope for this iteration of the blueprint.

Additional Context

Why not Kafka?

Given our needs to queue/buffer data durably, Apache Kafka comes as an obvious first choice. However, given the operational & distribution complexity around running Kafka especially as we shift focus towards running GitLab as cloud-native deployments, it becomes less favorable for our purposes. Following are some of our past discussions around the challenges Kafka brings:

- Support for Kafka across deployment-environments is non-existent.
- Kafka can be cost-prohibitive regardless of scale.
- Kafka can be operationally intensive.
- Non-trivial effort to replace current usage.

Key considerations when comparing Kafka with NATS

Feature	Comparison
Architecture	Kafka is a larger, distributed event streaming system while NATS is comparatively lightweight & high-performance messaging system. While Kafka is optimized for publish-subscribe usage, NATS allows all publish-subscribe, request-reply and data queueing patterns of usage.
Operations	Kafka requires Zookeeper/KRaft for coordination across partitions/topics/brokers while NATS ships as a single-binary with no external dependencies. Extending Kafka clusters warrants rebalancing partitions across brokers while NATS allows horizontally adding nodes more seamlessly. Kafka also warrants running it on a JVM while NATS can run natively on the given host.
Deployments	While Kafka can run cloud-natively within Kubernetes, it is comparatively more resource-intensive and comes with larger support overheads as compared to NATS with fewer components to operate.
Availability & Distribution	Ensuring Kafka is available across all our deployment-environments is challenging work, especially on smaller reference architectures given Kafka's cost-prohibitive nature even at small cluster-topologies. NATS on the other hand can be run with minimal overhead, be deployed in close proximity to GitLab installations with much smaller distribution effort.

SECTION: engineering/architecture/design-documents/gitlab_ml_experiments/_index.md

{{< engineering/design-document-header >}}

This document is an abbreviated proposal for Service-Integration to allow teams within GitLab to rapidly build new application features that leverage AI, ML, and data technologies.

Executive Summary

This document proposes a service-integration approach to setting up infrastructure to allow teams within GitLab to build new application features that leverage AI, ML, and data technologies at a rapid pace. The scope of the document is limited specifically to internally hosted features, not third-party APIs. The current application architecture runs most GitLab application features in Ruby. However, many ML/AI experiments require different resources and tools, implemented in different languages, with huge libraries that do not always play nicely together, and have different hardware requirements. Adding all these features to the existing infrastructure will increase the size of the GitLab application container rapidly, resulting in slower startup times, increased number of dependencies, security risks, negatively impacting development velocity, and increasing complexity due to different hardware requirements. As an alternative, the proposal suggests adding services to avoid overloading GitLabs main workloads. These services will run independently with isolated resources and dependencies. By adding services, GitLab can maintain the availability and security of GitLab.com, and enable engineers to rapidly iterate on new ML/AI experiments.

Scope

The infrastructure, platform, and other changes related to ML/AI experiments is broad. This blueprint is limited specifically to the following scope:

1. Production workloads, running (directly or indirectly) as a result of requests into the GitLab application (gitlab.com), or an associated subdomains (for example, codesuggestions.gitlab.com).
1. Excludes requests from the GitLab application, made to third-party APIs outside of our infrastructure. From an Infrastructure point-of-view, external AI/ML API requests are no different from other API (non ML/AI) requests and generally follow the existing guidelines that are in place for calling external APIs.
1. Excludes training and tuning workloads not directly connected to our production workloads. Training and tuning workloads are distinct from production workloads and will be covered by their own blueprint(s).

Running Production ML/AI experiment workloads

Why Not Simply Continue To Use The Existing Application Architecture?

Let's start with some background on how the application is deployed:

1. Most GitLab application features are implemented in Ruby and run in one of two types of Ruby deployments: broadly Rails and Sidekiq (although we do partition this traffic further for different workloads).

1. These Ruby workloads have two main container images gitlab-webservice-ee and gitlab-sidekiq-ee. All the code, libraries, binaries, and other resources that we use to support the main Ruby part of the codebase are embedded within these images.
1. There are thousands of pods running these containers in production for GitLab.com at any moment in time. They are started up and shut down at a high rate throughout the day as traffic demands on the site fluctuate.
1. For most new features developed, any new supporting resources need to be added to either one, or both of these containers.

\
source

Many of the initial discussions focus on adding supporting resources to these existing containers (example).
Choosing this approach would have many downsides, in terms of both the velocity at which new features can be iterated on, and in terms of the availability of GitLab.com.

Many of the AI experiments that GitLab is considering integrating into the application are substantially different from other libraries and tools that have been integrated in the past.

1. ML toolkits are implemented in a plethora of languages, each requiring separate runtimes. Python, C, C++ are the most common, but there is a long tail of languages used.
1. There are a very large number of tools that we're looking to integrate with and no single tool will support all the features that are being investigated. Tensorflow, PyTorch, Keras, Scikit-learn, Alpaca are just a few examples.
1. These libraries are huge. Tensorflow's container image with GPU support is 3GB, PyTorch is 5GB, Keras is 300MB. Prophet is ~250MB.
1. Many of these libraries do not play nicely together: they may have dependencies that are not compatible, or require different versions of Python, or GPU driver versions.

It's likely that in the next few months, GitLab will experiment with many different features, using many different libraries.

Trying to deploy all of these features into the existing infrastructure would have many downsides:

1. The size of the GitLab application container would expand very rapidly as each new experiment introduces a new set of supporting libraries, each library is as big, or bigger, than the existing GitLab application within the container.
1. Startup times for new workloads would increase, potentially impacting the availability of GitLab.com during high-traffic periods.
1. The number of dependencies within the container would increase rapidly, putting pressure on the engineering teams to keep ahead of exploits and vulnerabilities.

1. The security attack surface within the container would be greatly increased with each new dependency. These containers include secrets which, if leaked via an exploit would need costly application-wide secret rotation to be done.
1. Development velocity will be negatively impacted as engineers work to avoid dependency conflicts between libraries.
1. Additionally there may be extra complexity due to different hardware requirements for different libraries with appropriate drivers etc for GPUs, TPUs, CUDA versions, etc.
1. Our Kubernetes workloads have been tuned for the existing multithreaded Ruby request (Rails) and message (Sidekiq) processes. Adding extremely resource-intensive applications into these workloads would affect unrelated requests, starving requests of CPU and memory and requiring complex tuning to ensure fairness. Failure to do this would impact our availability of GitLab.com.

\
source

Proposal: Avoid Overfilling GitLabs Application Containers with Service-Integration

GitLab.com migrated to Kubernetes several years back, but for numerous good reasons, the application architecture deployed for GitLab.com remains fairly simple.

Instead of embedding these applications directly into the Rails and/or Sidekiq containers, we run them as small, independent Kubernetes deployments, isolated from the main workload.

\
source

The service-integration approach has already been used for the
GitLab Duo Suggested Reviewers feature
that has been deployed to GitLab.com.

This approach would have many advantages:

1. Componentization and Replaceability: some of these AI feature experiments will likely be short-lived. Being able to shut them down (possibly quickly, in an emergency, such as a security breach) is important. If they are terminated, they are less likely to leave technical debt behind in our main application workloads.
1. Security Isolation: experimental services can run with access to a minimal set of secrets, or possibly none. Ideally, the services would be stateless, with data being passed in, processed, and returned to the caller without access to PostgreSQL or other data sources. In the event of a remote code exploit or other security breach, the attacker would have limited access to sensitive data.

    1. In lieu of direct access to the main or CI Postgres clusters, services would be provided with access to the internal GitLab API through a predefined internal URL. The platform should provide instrumentation and monitoring on this address.
    1. In future iterations, but out of scope for the initial delivery, the platform could facilitate automatic authentication against the internal API, for example by managing and injecting short-lived API tokens into internal API calls, or OIDC etc.
1. Resource Isolation: resource-intensive workloads would be isolated to individual containers. OOM failures would not impact requests outside of the experiment. CPU saturation would not slow down unrelated requests.
1. Dependency Isolation: different AI libraries will have conflicting dependencies. This will not be an issue if they're run as separate services in Kubernetes.
1. Container Size: the size of the main application containers is not drastically increased, placing a burden on the application.
1. Distribution Team Bottleneck: The Distribution team avoids becoming a bottleneck as demands for many different libraries to be included in the main application containers increase.
1. Stronger Ownership of Workloads: teams can better understand how their workloads are running as they run in isolation.

However, there are several outstanding questions:

1. Availability Requirements: would experimental services have the same availability requirements (and alerting requirements) as the main application?
1. Oncall: would teams be responsible for handling pager alerts for their services?
1. Support for non-SAAS GitLab instances: initially all experiments would target GitLab.com, but eventually we may need to consider how to support other instances.
    1. There are three possible modes for services:
        1. M1: GitLab.com only: only GitLab.com supports the service.
        1. M2: SAAS-hosted for use with self-managed instance and instance-hosted: a singular SAAS-hosted service supports self-managed instances and GitLab.com. This is similar to the GitLab Plus proposal.
        1. M3: Instance-hosted: each instance has a copy of the service. GitLab.com has a copy for GitLab.com. Self-managed instances host their copy of the service. This is similar to the container registry or Gitaly today.
    1. Initially, most experiments will probably be option 1 but may be promoted to 2 or 3 as they mature.
1. Promotion Process: ML/AI experimental features will need to be promoted to non-experimental status as they mature. A process for this will need to be established.

Proposed Guidelines for Building ML/AI Services

1. Avoid adding any large ML/AI libraries needed to support experimentation to

   the main application.
1. Create an platform to support individual ML/AI experiments.
1. Encourage supporting services to be stateless (excluding deployed models and other resources generated during ML training).
1. ML/AI experiment support services must not access main application datastores, including but not limited to main PostgreSQL, CI PostgreSQL, and main application Redis instances.
1. In the main application, client code for services should reside behind a feature-flag toggle, for fine-grained control of the feature.

Technical Details

Some points, in greater detail:

Traffic Access

1. Ideally these services should not be exposed externally to Internet traffic: only internally to our existing Rails and Sidekiq workloads should be routed.
   1. For services intended to run at M2: "SAAS-hosted for use with self-managed instance and instance-hosted", we would expect to migrate the service to a public endpoint once sufficient security review has been performed.

Platform Requirements

In order to quickly deploy and manage experiments, an minimally viable platform will need to be provided to stage-group teams. The technical implementation details of this platform are out of scope for this blueprint and will require their own blueprint (to follow).

However, Service-Integration will establish certain necessary and optional requirements that the platform will need to satisfy.

Ease of Use, Ownership Requirements

ID	Required	Detail	Epic/Issue	Done?
R100	Required	The platform should be easy to use: imagine Heroku with GitLab Production Readiness-approved defaults.	Runway to [BETA] : Increased Adoption and Self Service	{dotted-circle} No
R110	Required	With the exception of an Infrastructure-led onboarding process, services are owned, deployed and managed by stage-group teams. In other words,services follow a "You Build It, You Run It" model of ownership.	[Paused] Discussion: Tiered Support Model for Runway	{dotted-circle} No
R120	Required	Programming-language agnostic: no requirements for services. Services should be packaged as container images.	Runway to [BETA] : Increased Adoption and Self Service	{dotted-circle} No
R130	Recommended	Each service should be evaluated against the GitLab.com Service Maturity Model.	Discussion: Introduce an 'Infrastructure Well-Architected Service Framework'	{dotted-circle} No
R140	Recommended	Services using the platform have expedited production-readiness

processes. Production-readiness requirements graded by service maturity: low-traffic, low-maturity experimental services will have lower requirement thresholds than more mature services. By default, the platform should provide services with defaults that would pass production-readiness review for the lowest service maturity-level. At introduction, lowest maturity services can be deployed without production readiness, provided the meet certain automatically validated requirements. This removes Infrastructure gate-keeping from being a blocker to experimental service delivery. | | |

Observability Requirements

ID	Required	Detail	Epic/Issue	Done?
R200	Required	The platform must provide SLIs for services out-of-the-box. - While it is recommended that services expose internal metrics, it is not mandatory. The platform will provide monitoring from the load-balancer. This is to speed up deployment by removing barriers to experimentation. - For services that provide internal metrics scrape endpoints, the platform must be configurable to collect these. - The platform must provide generic load-balancer level SLIs for all services. Service owners must be able to select from constructing SLIs from internal application metrics, the platform-provided external SLIs, or a combination of both.	Observability: Default Metrics, Observability: Custom Metrics	{check-circle} Yes
R210	Required	Observability dashboards, rules, alerts (with per-term routing) must be generated from a manifest.	Observability: Metrics Catalog	{check-circle} Yes
R220	Required	Standardized logging infrastructure. - Mandate that all logging emitted from services must be Structured JSON. Text logs are permitted but not recommended. - See Common Service Libraries for more details of building common SDKs for observability.	Observability: Logs in Elasticsearch for model-gateway, Observability: Runway logs available to users	

Deployment Requirements

ID	Required	Detail	Epic/Issue	Done?
R300	Required	No secrets stored in CI/CD. - Authentication with Cloud Provider Resources should be exclusively via OIDC, managed as part of the platform. - Secrets should be stored in the Infrastructure-provided Hashicorp Vault for the environment and passed to applications through files or environment variables. - Generation and management of service account tokens should be done declaratively, without manual interaction.	Secrets Management	{dotted-circle} No
R310	Required	Multiple environment should be supported, eg Staging and Production.		{check-circle} Yes
R320	Required	The platform should be cost-effective. Kubernetes clusters should support multiple services and teams.		
R330	Recommended	Gradual rollouts, rollbacks, blue-green deployments.		
R340	Required	Services should be isolated from one another.		
R350	Recommended	Services should have the ability to specify node characteristic requirements (eg, GPU).		
R360	Required	Developers should not need knowledge of Helm, Kubernetes, Prometheus in order to deploy. All required values are configured and validated in project-hosted manifest before generating Kubernetes manifests, Prometheus rules, etc.		

| R370 | | Initially services should be synchronous only - using REST or GRPC requests. This does not however preclude long-running HTTP(s) requests, for example long-polling or Websocket requests. | | |
| R390 | | Each service hosted in its own GitLab repository with deployment manifest stored in the repository. Continuous deployments that are initiated from the CI pipeline of the corresponding GitLab repository. | | |

Security Requirements

ID	Required	Detail	Epic/Issue	Done?
R400		Stateful services deployed on the platform that utilize their own stateful storage (for example, custom deployed Postgres instance), must not store application security tokens, cloud-provider service keys or other long-lived security tokens in their stateful stores.		
R410		Long-lived shared secrets are discouraged, and should be referenced in the service manifest as such, to allow for accounting and monitoring.		
R420		Services using long-lived shared secrets should ensure that secret rotation can take place without downtime. During a rotation, old and new generations of secrets should pass authentication, allowing gradual roll-out of new secrets.		

Common Service Libraries

ID	Required	Detail	Epic/Issue	Done?
R500	Required	Experimental services would be required to adopt and use LabKit (for Go services), or LabKit-Ruby for observability, context, correlation, FIPs verification, etc. At present, there is no LabKit-Python library, but some experiments will run in Python, so building a library to providing observability, context, correlation services in Python will be required.	Scalability: Labkit as the in-application platform toolkit	

SECTION: engineering/architecture/design-documents/gitlab_rag/_index.md

{{< engineering/design-document-header >}}

Goals

The goal of this blueprint is to describe viable options for RAG at GitLab across deployment types. The aim is to describe RAG implementations that provide our AI features·and by extension our customers·with best-in-class user experiences.

Status of RAG at GitLab

Feature	Current	Deprecated	Ongoing	Links
RAG for Duo Chat: Documentation question-answering	Vertex AI Search Released in %17.0	Postgres with PGVector extension Released in %16.0, deprecated in %17.0	Once AI Context

Abstraction layer is done, embeddings can be moved | Gitlab Duo RAG blueprint |
| RAG for Search features: hybrid issues and epics search | Elasticsearch Released in %17.6 | |
| Hybrid issue search epic |
| Future RAG features | | | AI Context Abstraction layer will support embedding storage and retrieval for GitLab data on Elasticsearch, OpenSearch or postgres | AI Context Abstraction layer blueprint AI Context Epic |

Overview of RAG

RAG, or Retrieval Augmented Generation, involves several key process blocks:

- Input Transformation: This step involves processing the user's input, which can vary from natural language text to JSON or keywords. For effective query construction, we might utilize Large Language Models (LLMs) to format the input into a standard expected format or to extract specific keywords.
- Retrieval: Here, we fetch relevant data from specified data sources, which may include diverse storage engines like vector, graph, or relational databases. It's crucial to conduct data access checks during this phase. After retrieval, the data should be optimized for LLMs through post-processing to enhance the quality of the generated responses.
- Generation: This phase involves crafting a prompt with the retrieved data and submitting it to an LLM, which then generates an AI-powered response.

(Image from Deconstructing RAG)

Challenges of RAG

Data for LLMs

Ensuring data is optimized for LLMs is crucial for consistently generating high-quality AI responses. Several challenges exist when providing context to LLMs:

- Long Contexts: Extensive contexts can degrade LLM performance, a phenomenon known as the Lost in the Middle problem. Employing Rerankers can enhance performance but may also increase computational costs due to longer processing times.
- Duplicate Contents: Repetitive content can reduce the diversity of search results. For instance, if a semantic search yields ten results indicating "Tom is a president" but the eleventh reveals "Tom lives in the United States," solely using the top ten would omit critical information. Filtering out duplicate content, for example, through Maximal Marginal Relevance (MMR), can mitigate this issue.
- Conflicting Information: Retrieving conflicting data from multiple sources can lead to LLM "hallucinations." For example, mixing sources that define "RAG" differently can confuse the LLM. Careful source selection and content curation are essential.
- Irrelevant Content: Including irrelevant data can negatively impact LLM

performance. Setting a threshold for relevance scores or considering that certain irrelevant contents might actually enhance output quality are strategies to address this challenge.

It's highly recommended to evaluate the optimal data format and size for maximizing LLM performance, as the effects on performance and result quality can vary significantly based on the data's structure.

References:

- Benchmarking Methods for Semi-Structured RAG
- Edge cases of semantic search

Regenerating Embeddings

The AI field is evolving rapidly and new models and approaches seem to appear daily that could improve our users' experience, we want to conscious of model switching costs. If we decide to swap models or change our chunking strategy (as two examples), we will need to wipe our existing embeddings and do a full replacement with embeddings from the new model or with the new text chunks, etc. Factors to consider which could trigger the need for a full regeneration of embeddings for the affected data include:

- A change in the optimal text chunk size
- A change in a preprocessing step which perhaps adds new fields to a text chunk
- Exclusion of content, such as the removal of a field that was previously embedded
- Addition of new metadata that needs to be embedded

Multi-source Retrieval

Addressing complex queries may require data from multiple sources. For instance, queries linking issues to merge requests necessitate fetching details from both. GitLab Duo Chat, utilizing the ReACT framework, sequentially retrieves data from PostgreSQL tables, which can prolong the retrieval process due to the sequential execution of multiple tools and LLM inferences.

Searching for Data

Choosing the appropriate search method is pivotal for feature design and UX optimization. Here are common search techniques:

Semantic Search using embeddings

Semantic search shines when handling complex queries that demand an understanding of the context or intent behind the words, not just the words themselves. It's particularly effective for queries expressed in natural language, such as full sentences or questions, where the overall meaning outweighs the importance of specific keywords. Semantic search excels at providing thorough coverage of a topic, capturing related concepts that may not

be directly mentioned in the query, thus uncovering more nuanced or indirectly related information.

In the realm of semantic search, the K-Nearest Neighbors (KNN) method is commonly employed to identify data segments that are semantically closer to the user's input by using embeddings. To measure the semantic proximity, various methods are used:

- Cosine Similarity: Focuses solely on the direction of vectors.
- L2 Distance (Euclidean Distance): Takes into account both the direction and magnitude of vectors. These vectors, known as "embeddings," are created by processing the data source through an embedding model. Currently, in GitLab production, we utilize the textembedding-gecko model provided by Vertex AI. However, there might be scenarios where you consider using alternative embedding models, such as those available on HuggingFace, to reduce costs. Opting for different models requires comprehensive evaluation and consultation, particularly with the legal team, to ensure the chosen model's usage complies with GitLab policies. See the Security, Legal, and Compliance section for more details. It's also important to note that multilingual support can vary significantly across different embedding models, and switching models may lead to regressions.

For large datasets, it's advisable to implement indexes to enhance query performance. The HNSW (Hierarchical Navigable Small World) method, combined with approximate nearest neighbors (ANN) search, is a popular strategy for this purpose. For insights into HNSW's effectiveness, consider reviewing benchmarks on its performance in large-scale applications.

Due to the existing framework and scalability of Elasticsearch, embeddings will be stored on Elasticsearch for large datasets such as issues, merge requests, etc. This will be used to perform Hybrid Search but will also be useful for other features such as finding duplicates, similar results or categorizing documents.

Keyword Search

Keyword search is the go-to method for straightforward, specific queries where users are clear about their search intent and can provide precise terms or phrases. This method is highly effective for retrieving exact matches, making it suitable for searches within structured databases or when looking for specific documents, terms, or phrases.

Keyword search operates on the principle of matching the query terms directly with the content in the database or document collection, prioritizing results that have a high frequency of the query terms. Its efficiency and directness make it particularly useful for situations where users expect quick and precise results based on specific keywords or phrases.

Elasicsearch uses a BM25 algorigthm to perform keyword search.

If one of the existing indexed document types
is not covered, a new document type can be added.

Hybrid Search

Hybrid search combines the depth of semantic search with the precision of keyword search, offering a comprehensive search solution that caters to both context-rich and specific queries. By running both semantic and keyword searches simultaneously, it integrates the strengths of both methods·semantic search's ability to understand the context and keyword search's precision in identifying exact matches.

The results from both searches are then combined, with their relevance scores normalized to provide a unified set of results. This approach is particularly effective in scenarios where queries may not be fully served by either method alone, offering a balanced and nuanced response to complex search needs. The computational demands of kNN searches, which are part of semantic search, are contrasted with the relative efficiency of BM25
keyword searches, making hybrid search a strategic choice for optimizing performance across diverse datasets.

The first hybrid search scope is for issues which combines keyword search with kNN matches using embeddings.

Code Search

Like the other data types above, a source code search task can use different search types, each more suited to address different queries.

Two code searches are available: Elasticsearch and Zoekt.

Elasticsearch provides blob search which supports Advanced Search Syntax.

Zoekt is employed on GitLab.com to provide
exact match keyword search and regular expression search capabilities for source code.

Semantic search and hybrid search functionalities are yet to be
implemented for code.

ID Search

Facilitates data retrieval using specific resource IDs, such as issue links. For example retrieving data from the specified resource ID, such as an Issue link or a shortcut. See ID search for more information.

Knowledge Graph

Knowledge Graph search transcends the limitations of traditional search methods by leveraging the interconnected nature of data represented in graph form.

Unlike semantic search, which focuses on content similarity, Knowledge Graph search understands and utilizes the relationships between different data points, providing a rich, contextual exploration of data.

This approach is ideal for queries that benefit from understanding the broader context or the interconnectedness of data entities. Graph databases store relationships alongside the data, enabling complex queries that can navigate these connections to retrieve highly contextual and nuanced information.

Knowledge Graphs are particularly useful in scenarios requiring deep insight into the relationships between entities, such as recommendation systems, complex data analysis, and semantic querying, offering a dynamic way to explore and understand large, interconnected datasets.

Security, Legal and Compliance

Data access policy

The retrieval process must comply with the
GitLab Data Classification Standard.
If the user doesn't have access to the data, GitLab will not fetch the data for building a prompt.

For example:

- When the data is GitLab Documentation (GREEN level), the data can be fetched without authorizations.
- When the data is customer data such as issues, merge requests, etc (RED level), the data must be fetched with proper authorizations based on permissions and roles.

If you're proposing to fetch data from an external public database
(e.g. fetching data from arxiv.org so the LLM can answer questions about quantitative biology), please conduct a thorough review to ensure the external data isn't inappropriate for GitLab to process.

Data usage

Using a new embedding model or persisting data into a new storage would require legal reviews. See the following
links for more information:

- Data privacy
- Data retention
- Training data

Evaluation

Evaluation is a crucial step in objectively determining the quality of the retrieval process. Tailoring the retrieval process based on specific user feedback can lead to biased optimizations, potentially causing regressions for

other users. It's essential to have a dedicated test dataset and tools for a comprehensive quality assessment. For assistance with AI evaluation, please reach out to the AI Model Validation Group.

Before Implementing RAG

Before integrating Retrieval Augmented Generation (RAG) into your system, it's important to evaluate whether it enhances the quality of AI-generated responses. Consider these essential questions:

- What does typical user input look like?
  - For instance, "Which class should we use to make an external HTTP request in this repository?"
- What is the desired AI-generated response?
  - Example: "Within this repository, Class-A is commonly utilized for..."
- What are the current responses from LLMs? (This helps determine if the necessary knowledge is already covered by the LLM.)
  - Example: Receiving a "Sorry, I don't have an answer for that." from the Anthropic Claude 2.1 model.
- What data is required in the LLM's context window?
  - Example: The code for Class-A.
- Consider the current search method used for similar tasks. (Ask yourself: How would I currently search for this data with the tools at my disposal?)
  - Example: Navigate to the code search page and look for occurrences of "http."
- Have you successfully generated the desired AI response with sample data? Experiment in a third-party prompt playground or Google Colab to test.
- If contemplating semantic search, it's highly recommended that you develop a prototype first to ensure it meets your specific retrieval needs. Semantic search may interpret queries differently than expected, especially when the data source lacks natural language context, such as uncommented code. In such cases, semantic search might not perform as well as traditional keyword search methods. Here's an example prototype that demonstrates semantic search for CI job configurations.

Evaluated Solutions

The following solutions have been validated with PoCs to ensure they meet the basic requirements of vector storage and retrieval for GitLab Duo Chat with GitLab documentation. Click the links to learn more about each solutions attributes that relate to RAG:

- PostgreSQL with PGVector
- Elasticsearch
- Google Vertex

To read more about the GitLab Duo Chat PoCs conducted, see:

- PGVector PoC
- Elasticsearch PoC
- Google Vertex PoC

SECTION: engineering/architecture/design-documents/gitlab_rag/elasticsearch.md

Elasticsearch is a search engine and data store which allows generating, storing and querying vectors and performing keyword and semantic search at scale.

Elasticsearch employs a distributed architecture, where data is stored across multiple nodes. This allows for parallel processing of queries, ensuring fast results even with massive datasets.

Using Elasticsearch as a vector store

Elasticsearch can be used to store embedding vectors up to 4096 dimensions and find the closest neighbours for a given embedding.

Licensing

Does not require a paid license.

Indexing embeddings

For every document type (e.g. gitlabdocumentation), an index is created and stores the original source, embeddings and optional metadata such as URL. An initial backfill is required to index all current documents and a process to upsert or delete documents as the source changes.

For GitLab Duo Documentation, the current async process for generating and storing embeddings in the embeddings database can be altered to index into Elasticsearch.

Using the Advanced Search framework, database records are automatically kept up to date in Elasticsearch. Issue 442197 proposes changing the Elasticsearch framework to allow for other datasets to be indexed.

For documents with large sources that need to be split into chunks, nested kNN search can be used whereby a single top-level document contains nested objects each with a source and embedding. This enables searching for the top K documents with the most relevant chunks. It is not suited for cases where the top k chunks need to be searched within a single document. In such cases, every chunk should be stored as a separate document.

Querying context-relevant information

A given question is passed to a model to generate embeddings. The vector is then sent to Elasticsearch to find the most relevant documents.

Generation

The N most relevant documents are added to a prompt which is sent to an LLM to generate an answer for the original question.

RAG in Elasticsearch using hosted models

Similar to the above but the question's embeddings are generated from within Elasticsearch.

Licensing

Requires a paid license on every cluster.

Model hosting

Requires model(s) used to be hosted on every cluster which adds effort and cost.

Elasticsearch supports the following models:

- ELSER (Elastic Learned Sparse Encoder): Built-in model provided by Elasticsearch used to generate text embeddings for semantic search.
- TensorFlow Models: Custom TensorFlow models can be deployed for semantic search using the ML APIs.
- Third-Party Models: Elasticsearch supports deploying models from Hugging Face and other providers. This provides access to a wider range of pre-trained models, but deployment and maintenance requires additional work.

Hybrid Search

Hybrid search combines text and semantic search to return the most revelant sources. A reranker could be used to combine the results from both methods.

Advanced text search features of Elasticsearch

1. Inverted Indexing: At its core, Elasticsearch relies on a powerful data structure called an inverted index. This index essentially flips the traditional approach, where each document contains a list of words. Instead, the inverted index catalogues every unique word across all documents and tracks where it appears in each one. This enables lightning-fast searches by finding relevant documents based on matching words instantly.

1. Advanced Text Analysis: Elasticsearch doesn't simply match whole words. It leverages text analyzers to break down and understand text intricacies. This includes handling:

  - Stemming and lemmatization: Reducing words to their root form (e.g., "running" and "ran" both matching "run").
  - Synonyms and related terms: Recognizing synonyms and similar words to expand search results.
  - Stop words: Ignoring common words like "the" and "a" that don't contribute much to meaning.
  - Custom analysis: Defining your own rules for specific domains or languages.

1. Powerful Query Capabilities: Elasticsearch goes beyond basic keyword searches. It supports complex queries using Boolean operators (AND, OR, NOT), proximity searches (finding words close together), fuzzy searches (handling typos), and more. You can also filter results based on other criteria alongside text matching.

Reranking

Elasticsearch currently supports Reciprocal rank fusion (RRF) which works out-the-box. They also released Learning to Rank which uses ML to improve ranking.

Running Elasticsearch

Elasticsearch is available on GitLab.com and can be integrated on Dedicated and Self-Managed instances. To use as a vector store only:

- Install Elasticsearch version 8.12 or upgrade to at least version 8.12.
- Add URL, Username and Password on the Advanced Search settings page: admin/applicationsettings/advancedsearch

After the integration is configured, instance admins don't need to do further work to use it as a vector store since the GitLab Elasticsearch framework handles setting mappings, settings and indexing data.

Supported dimensions

Elasticsearch can store up to 4096 dimensions and OpenSearch up to 16000 dimensions, compared to pgvector which can store up to 2000.

Limitations

Licensing

In order to use the ML capabilities offered by Elastic, every cluster has to have a valid license.

If Elastic is used only as a vector store and all embeddings generated outside of Elastic, a license is not required.

Adoption

The Elastic integration is available to all GitLab instances to unlock Advanced Search but not all instances have chosen to run the integration. There is also an additional cost for every instance to host the integration.

Performance and scalability

Elasticsearch is horizontally scalable and handles storing and querying at scale. An Elasticsearch cluster consists of multiple nodes each contributing resources.

Cost

Elastic Cloud pricing for GitLab Documentation vector storage is about $38 per month and the price scales with storage requirements.

Elasticseach vs. OpenSearch

Features

Both offer storing vector embeddings and similarity search (kNN).

Elasticsearch supports custom TensorFlow models which OpenSearch does not offer. Both offer pre-trained models.

The APIs for kNN searching differ slightly between the two platforms but work in the same way.

Supported platforms

Currently GitLab offers Advanced Search for both Elasticsearch and OpenSearch due to parity between the text search APIs. If both are supported for AI features, there would be a need to adapt to two different AI APIs.

PoC: Repository X Ray

To test the viability of Elasticsearch for generating embeddings, a PoC was done with Repository X Ray project.

Repository X Ray hasn't yet implemented any semantic seach and this section is based soely on a prototype implementation

- Statistics (as of February 2024):
  - Data type: JSON document with source code libraries descriptions in natural language
  - Data access level: Red (each JSON document belongs to specific project, and data access rules should adhere to data access rules configure for that project)
  - Data source: Repository X Ray report CI artifact
  - Data size: N/A
  - Example of user input: '' generate function that fetches sales report for vendor from App Store''
  - Example of expected AI-generated response:

```
  python
  def salesreports(vendorid)\n  appstoreconnect.salesreports(\n  filter: {\n    reporttype: 'SALES',\n    reportsubtype: 'SUMMARY',\n    frequency: 'DAILY',
    vendornumber: '123456'\n  }\n)\nend
```

Synchronizing embeddings with data source

In a similar manner as with the documentation example Repository X Ray report data is a derivative. It uses an underlying repository source code as a base, and it must be synchronised with it, whenever any changes to the source code occurs.

Right now there is no synchronisation mechanism that includes embeddings and vector storage. However there is an existing pipeline that generates and stores Repository X Ray reports.

The ingestion pipeline is performed in following steps:

1. A CI X Ray scanner job is triggered - a documentation page suggest limiting this job to be executed only when changes occur to the main repository branch. However repository maintainers may configure trigger rules differently.
   - An X Ray scanner locates and process one of the supported dependencies files, producing JSON report files.
1. After the X Ray scanner job finishes successfully, a background job is triggered in GitLab Rails monolith that imports JSON report into Projects::XrayReport.
   - There can be only one Repository X Ray report per project in the scope of programming language, duplicated records are being upserted during import process

As of today, there are 84 rows on xrayreports table on GitLab.com.

Retrieval

After Repository X Ray report gets imported, when IDE extension sends request for a code generation,
Repository X Ray report is retrieved in the following steps:

1. GitLab Rails monotlith fetches corresponding xrayreports record from main database. xrayreports records are filiterd based on projectid foreign key, and lang columns.
1. From retrieved record first 50 dependencies are being added into a prompt that is forwarded to AI Gateway

Current state overview

mermaid
sequenceDiagram
    actor USR as User
    participant IDE
    participant GLR as GitLabRails
    participant RN as GitLabRunner
    participant PG as GitLabPsqlMainDB
    participant AIGW as AIGateway
    USR->>+GLR: commits changes to Gemfile.lock
    GLR->>RN: triggers Repository X Ray CI scanner job
    RN->>GLR: Repository X Ray report
    GLR->>GLR: triggers Repository X Ray ingestion job
    GLR->>-PG: upserts xrayreports record

    USR->>+IDE: types: "35; generate function that fetches sales report for vendor from App Store"
    IDE->>+GLR: trigger code generation for line "35; generate function
    GLR->>PG: fetch X Ray report for project and language
    PG->>GLR: xrayreports record
    GLR->>GLR: include first 50 entities from xray report into code generation prompt

GLR->>-AIGW: trigger code generation "35; generate function

Embeddings prospect application

As described in retrieval section above, currently Repository X Ray reports follow very naive approach, that does not include any metric for assessing relevance between Repository X Ray report content and
user instruction. Therefore applying embeddings and semantic search to X Ray report has a high potential of improving results by selecting limited set of related entries from Repository X Ray report based on user instruction.

To achieve that embeddings should be generated during Repository X Ray ingestion. Additionally an user instruction should be turned into embeddings vector to perform semantic search over stored Repository X Ray report data during retrieval process.

Elasticsearch and PGVector comparison

Following paragraph is a result of PoC work.

From a product feature implementation point of view both solutions seems as viable ones, offering in the current state all necessary tools to support the product features requirements. Given the Elasticsearch built in capabilities it is acknowledged that it might bring better long term support, enabling more powerful RAG solution in the future than pgvector based ones.

The current Elasticsearch integration only indexes ActiveRecord models and source code from Git repositories. Further work required to build more generic abstractions to index other data (eg. X-Ray reports) has been defined by issue 442197.
To prevent suboptimal workarounds of existing limitation which will create technical debt, it is advised that issue 442197 is completed before Elasticsearch is selected as main vector storage for RAG.

SECTION: engineering/architecture/design-documents/gitlab_rag/postgresql.md

This page explains how to retrieve data from PostgreSQL for RAG.

Semantic search

Overview

1. Install PgVector extension to the PostgreSQL database.
1. Add a vector column to a new or existing table.
1. Data <=> Embedding synchronization
   1. Load data which you want to search from.
   1. Pass the data to an embedding model and get an vector.
   1. Set the vector to the vector column.
1. Retrieval

1. Pass the user input to an embedding model and get an vector.
1. Get the nearest neighbors to the user input vector e.g. SELECT FROM atable ORDER BY vectorcolumn <-> '<user-input-vector>' LIMIT 5;

Vector store with PgVector

To store the embeddings for semantic search, we need to add a vector store in GitLab PostgreSQL.
This vector store can be added by installing PgVector extension (Postgres 12+ is required).
A vector store is currently running on GitLab.com and it's separately hosted from the main/CI databases.

Our current architecture of having a separate database for embeddings is probably ideal. We don't gain much by combining them and, as PGVector is all new and will likely require a lot of experimenting to get performance at scale (today we only have a tiny amount of data in it), we'll have a lot more options to experiment with without impacting overall GitLab.com stability (if PGVector is on a separate database). Having a separate database is recommended because it allows for experimentation without impacting performance of the main database.

Limitations

- It could be locked down to a specific embedding model, because you must specify the dimensions of the vector column.
- Vectors with up to 2,000 dimensions can be indexed.

Performance and scalability implications

- Is there any guidance on how much data we can add to the PostgreSQL (regardless of the vector data or normal data)?
  - Not really, as we do not usually just add data to the database, but rather it's a result of the instance being used. I don't see any specific storage requirements. If the existing vertexgitlabdocs table size is a good indicator, we probably can add this without causing much trouble, though having an option to opt-in or opt-out is preferable.

Availability

- PostgreSQL is availble in all GitLab installations (both CNG and Omnibus).
- Most major cloud providers have added PgVector to their offerings by now: Google Cloud SQL and Alloy DB, DigitalOcean, AWS RDS and Aurora, Azure Flexible and Cosmos, etc. There might be a case where customers would need to upgrade to versions that support PGVector.

ID search

Overview

1. Execute a few-shot prompts to extract a resource identifier from the user input.
   - e.g. When user asks Can you summarize 12312312?, ResourceIdentifier is 12312312 as a GitLab-Issue.
1. Retrieve the record from the PostgreSQL. e.g. Issue.find(12312312)
1. Check if the user can read the resource.

1. Build a prompt with the retrieved data and passing it to an LLM to get a AI-generated response.

PoC: Repository X Ray

Repository X Ray hasn't yet implemented any semantic seach and this section is based soely on a prototype implementation

- Statistics (as of February 2024):
  - Date type: JSON document with source code libraries desctiptions in natural language
  - Date access level: Red (each JSON document belongs to specific project, and data access rules should adhere to data access rules configure for that project)
  - Data source: Repository X Ray report CI artifact
  - Data size: N/A
  - Example of user input: " generate function that fetches sales report for vendor from App Store"
  - Example of expected AI-generated response:

```
python
def salesreports(vendorid)\n  appstoreconnect.salesreports(\n  filter: {\n    reporttype: 'SALES',\n    reportsubtype: 'SUMMARY',\n    frequency: 'DAILY',
    vendornumber: '123456'\n  }\n)\nend
```

Synchronizing embeddings with data source

In symilar manner as with the documentation example Repository X Ray report data is a derivative. It uses an underlaying repository source code as a base,
and it must be synchronised with it, whenever any changes to the source code occurs.

Right now there is no synchronisation mechanism that includes embeddings and vector storage. However there is an existing pipeline that generates and stores Repository X Ray reports

The ingestion pipeline is performed in following steps:

1. A CI X Ray scanner job is triggered - a documentation page suggest limiting this job to be executed only when changes occur to the main repository branch. However repository maintainers may configure trigger rules differently.
   1. An X Ray scanner locates and process one of the supported dependencies files, producing JSON report files
1. After the X Ray scanner job finishes successfully, a background job is triggered in GitLab Rails monolith that imports JSON report into Projects::XrayReport
   1. There can be only one Repository X Ray report per project in the scope of programming language, duplicated records are being upserted during import process

As of today, there are 84 rows on xrayreports table on GitLab.com.

Retrieval

After Repository X Ray report gets imported, when IDE extension sends request for a code

generation, Repository X Ray report is retrieved, in following steps

1. Fetch embedding of the user input from textembedding-gecko model (768 dimensions).
1. Query to vertexgitlabdocs table for finding the nearest neighbors. For example:

```
sql
SELECT
FROM vertexgitlabdocs
ORDER BY vertexgitlabdocs.embedding <=> '[vectors of user input]'              -- nearest neighbors by cosine distance
LIMIT 10
```

1. GitLab Rails monotlith fetches corresponding xrayreports record from main database. xrayreports records are filiterd based on projectid foreign key, and lang columns.
1. From retrieved record first 50 dependencies are being added into a prompt that is forwarded to AI Gateway

Current state overview

```
mermaid
sequenceDiagram
    actor USR as  User
    participant IDE
    participant GLR as GitLabRails
    participant RN as GitLabRunner
    participant PG as GitLabPsqlMainDB
    participant AIGW as AIGateway
    USR->>+GLR: commits changes to Gemfile.lock
    GLR->>RN: triggers Repository X Ray CI scanner job
    RN->>GLR: Repository X Ray report
    GLR->>GLR: triggers Repository X Ray ingestion job
    GLR->>-PG: upserts xrayreports record

    USR->>+IDE: types: "35; generate function that fetches sales report for vendor from App Store"
    IDE->>+GLR: trigger code generation for line  "35; generate function
    GLR->>PG: fetch X Ray report for project and language
    PG->>GLR:  xrayreports record
    GLR->>GLR: include first 50 entities from xray report into code generation prompt
    GLR->>-AIGW: trigger code generation  "35; generate function
```

Embeddings prospect application

As described in retrieval section above, currently Repository X Ray reports follows very naive approach, that does not iclude any metric for assesing relevance between Repository X Ray report content and user instruction. Therefore applying embeddings and semantic search to X Ray report has a high potential of improving results by selecting limited set of related entries from Repository X Ray report based on user instruction.

To achieve that embeddings should be generated during Repository X Ray ingestion. Additionaly an user instruction should be turned into embeddings vector to perform semantic search over stored Repository X Ray report data during retrieval process.

SECTION: engineering/architecture/design-documents/gitlab_rag/vertex_ai_search.md

This page explains how to retrieve data from Google Vertex AI Search for RAG.

Overview

Some of our data are public resources that don't require data access check when retrieving. These data are often identical across GitLab instances so it's redundant to ingest the same data into every single database.
It'd be more efficient to serve the data from the single service.

We can use Vertex AI Search in this case.
It can search at scale, with high queries per second (QPS), high recall, low latency, and cost efficiency.

This approach allows us to minimize code that we can't update on a customer's behalf, which means avoiding hard-coding AI-related logic in the GitLab monolith codebase. We can retain the flexibility to make changes in our product without asking customers to upgrade their GitLab version.
This is same with the AI Gateway's design principle.

```
mermaid
flowchart LR
  subgraph GitLab managed
    subgraph AIGateway
       VertexAIClient["VertexAIClient"]
    end

    subgraph Vertex AI Search["Vertex AI Search"]
       subgraph SearchApp1["App"]
       direction LR
       App1DataStore(["BigQuery"])
       end
       subgraph SearchApp2["App"]
       direction LR
       App2DataStore(["Cloud Storage / Website URLs"])
       end
    end
  end

  subgraph SM or SaaS GitLab
    DuoFeatureA["Duo feature A"]
    DuoFeatureB["Duo feature B"]
  end
```

DuoFeatureA -- Semantic search --- VertexAIClient
DuoFeatureB -- Semantic search --- VertexAIClient
VertexAIClient -- Search from Gitlab Docs --- SearchApp1
VertexAIClient -- Search from other data store --- SearchApp2

Limitations

- Data must be GREEN level and publicly shareable.
  - Examples:
  - GitLab documentations (gitlab-org/gitlab/doc, gitlab-org/gitlab-runner/docs, gitlab-org/omnibus-gitlab/doc, etc)
  - Dynamically construct few-shot prompt templates with Example selectors.

IMPORTANT: We do NOT persist customer data into Vertex AI Search. See the other solutions for persisting customer data.

Performance and scalability implications

- GitLab-side: Vertex AI Search can search at scale, with high queries per second (QPS), high recall, low latency, and cost efficiency.
- GitLab-side: Vertex AI Search supports global and multi-region deployments.
- Customer-side: The outbound requests from their GitLab Self-managed instances could cause more network latency than retrieving from a local vector store.
  This latency issue is addressable by multi-region deployments.

Availability

- Customer-side: Air-gapped solutions can't be supported due to the required access to AI Gateway (cloud.gitlab.com).
  This concern would be negligible since GitLab Duo already requires the access.
- Customer-side: Since the service is the single point of failure, retrievers stop working when the service is down.

Cost implications

- GitLab-side: See Vertex AI Search pricing.
- Customer-side: No additional cost required.

Maintenance

- GitLab-side: GitLab needs to maintain the data store (e.g. Structured data in Bigquery or unstructured data in Cloud Storage). Google automatically detects the schema and indexes the stored data.
- Customer-side: No maintenance required.

SECTION: engineering/architecture/design-documents/gitlab_services/_index.md

{{< engineering/design-document-header >}}

Summary

To orthogonally capture the modern service-oriented deployment and environment managements, GitLab needs to have services as the first-class concept.
This blueprint outlines how the service and related entities should be built in the GitLab CD solution.

Motivation

As GitLab works towards providing a single platform for the whole DevSecOps cycle,
its offering should not stop at pipelines, but should include the deployment and release management, as well as
observability of user-developed and third party applications.

While GitLab offers some concepts, like the environment syntax in GitLab pipelines,
it does not offer any concept on what is running in a given environment. While the environment might answer the "where" is
something running, it does not answer the question of "what" is running there. We should introduce service and release artifact to answer this question. The Delivery glossary defines a service as

> a logical concept that is a (mostly) independently deployable part of an application that is loosely coupled with other services to serve specific functionalities for the application.

A service would connect to the SCM, registry or issues through release artifacts and would be a focused view into the environments where
a specific version of the given release artifact is deployed (or being deployed).

Having a concept of services allows our users to track their applications in production, not only in CI/CD pipelines. This opens up possibilities, like cost management.
The current work in Analyze:Observability could be integrated into GitLab if it supports services.

Goals

- Services are defined at the project level.
- A single project can hold multiple services.
- We should be able to list services at group and organization levels. We should make sure that our architecture is ready to support group-level features from day one. "Group-level environment views" are a feature customers are asking for many-many years now.
- Services are tied to environments. Every service might be present in multiple environments and every environment might host multiple services. Not every service is expected to be present in every environment and no environment is expected to host all the services.
- A service is a logical concept that groups several resources.
- A service is typically deployed independently of other services. A service is typically deployed in its entirety.
  - Deployments in the user interviews happened using CI via Helm, helmfiles or Flux and a HelmRelease.

  - Even in Kubernetes, there might be other tools (Kustomize, vanilla manifests) to deploy a service.
  - Outside of Kubernetes other tools might be used. e.g. Runway deploys using Terraform.
- We want to connect a deployment of a service to the MRs, containers, packages, linter results included in the release artifact.
- A service contains a bunch of links to external (or internal) pages.

src of the architecture diagram

(The dotted border for Deployment represents a projection to the Target Infrastructure)

Non-Goals

- Metrics related to a service should be customizable and configurable by project maintainers (developers?). Metrics might differ from service to service both in the query and in the meaning. (e.g. traffic does not make sense for a queue).
- Metrics should integrate with various external tools, like OpenTelemetry/Prometheus, Datadog, etc.
- We don't want to tackle GitLab observability solution built by Analyze:Observability. The proposal here should treat it as one observability integration backend.
- We don't want to cover alerting, SLOs, SLAs and incident management.
- Some infrastructures might already have better support within GitLab than others (Kubernetes is supported better than pure AWS). There is no need to discuss functionalities that we provide or plan to provide for Kubernetes and how to achieve feature parity with other infrastructures.
- Services can be filtered by metadata (e.g. tenant, region). These could vary by customer or even by group.

Proposal

Introduce a Service model. This is a shallow model that contains the following parameters:

- Name: The name of the service (e.g. Awesome API)
- Description: Markdown field. It can contain links to external (or internal) pages.
- (TBD) Metadata: User-defined key-Value pairs to label the service, which later can be used for filtering at group or project level.
  - Example fields:
  - Tenant: northwest
  - Component: Redis
  - Region: us-east-1
- (TBD) Deployment sequence: To allow the promotion from dev to staging to production.
- (TBD) Environment variables specific to services: Variables within an environment, variables should be definable for services as well.

DORA metrics

Users can observe DORA metrics through Services:

- Today, deployment frequency counts the deployments with an environmenttier=production or the

job name being prod or production.
- It should be clear for end-users. It can be a convention, like restricting a pipeline to a single environmenttier=production job or the first environmenttier=production per environment. To be defined later.

Aggregate environments and services at group level

At the group-level, GitLab fetches all of the project-level environments under the specific group,
and grouping it by the name of the environments. For example:

	Frontend service	Backend service
dev	Release Artifact v0.x	
development	Release Artifact v0.y	
production	Release Artifact v0.z	Release Artifact v1.x

Entity relationships

- Service and Environment has many-to-many relationship.
- Deployment and Release Artifact have many-to-one relationship (for a specific version of the artifact). Focusing on a single environment, Deployment and Release Artifact have a one-to-one relationship.
- Environment and Deployment has one-to-many relationship. This allows to show a deployment history (past and running, no outstanding, roll-out status might be included) by Environment.
- Environment and Release Artifact has many-to-many relationship through Deployment.
- Service and Release Artifact has many-to-many relationship. This allows to show a history of releases (past, running and outstanding) by service
- Release Artifact and Artifact have one-to-many relationship (e.g. chart as artifact => value as artifact => image as artifact).

mermaid
classDiagram
    Group "1" o-- "" Project : There may be multiple projects with services in a group
    Project "1" <.. "" Service : A service is part of a project
    Project "1" <.. "" Environment : An environment is part of project
    Environment "" .. "" Service : A service is linked to 1+ environments
    Service "1" <|-- "" ReleaseArtifact : A release artifact packages a specific version of a service
    ReleaseArtifact "1